

methods to help you: `.has_other_pages()`, `.has_next()`, `.has_previous()`, and `.next_page_number()`. Using these inside conditional blocks allows you to appropriately render links that point to other pages in the pagination result. Remember that object methods get accessed the same way as attributes in the template language, so you won't need any parenthesis.

Listing 4.12 shows the conditional block that goes on the bottom of `RiffMates/templates/musicians.html`. As the `musicians` view passes in `pages.object_list` as `musicians` in the context, the rest of the page can remain unchanged.

Listing 4.12 Musicians listing view supporting pagination

```
<!-- RiffMates/templates/musicians.html -->  
<!-- ... -->
```

Link to the musicians view (this page) with a query parameter for the previous page.

```
{% if page.has_other_pages %}  
    {% if page.has_previous %}  
        <a href="{% url 'musicians' %}?page={{page.previous_page_number}}"  
            >Prev</a> &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~  
    {% endif %}
```

And a query parameter for the next page

```
{% if page.has_next %}  
    <a href="{% url 'musicians' %}?page={{page.next_page_number}}"  
        >Next</a>  
    {% endif %}  
{% endif %}
```

With the changes in place, you have a paginated version of your musicians listing page. Run your development server and revisit the page. Try the URL with and without the `page` query parameter. You may also want to modify the `Paginator` object, changing how many items per page to see how the page changes.

Musicians are great, but to make a great tune, you need to group them together. To capture a band in your database, you need to create relations between musicians.

4.7 Model relations

A single object in the database will only get you so far. The *relational* in *relational database* is all about the connections between objects. There are two ways to relate objects to each other in a database. The first is to add a relationship column to your object table. The relationship column contains the ID of the related object, which is known as a *foreign key*. Foreign key relationships are one to many. This means multiple things can be related, but they are related through a single object. A music venue can have multiple rooms, each with its own schedule, but each room can only be associated with a single venue. That association between one venue and many rooms creates our one-to-many relationship.

The second way to interrelate objects in a database is to add a new table. The new table has the sole purpose of specifying object-to-object relationships. This kind of table has three columns: a unique ID, a foreign key to the first kind of object, and a foreign key to the second kind of object. Each row in the database expresses the relation between two objects. This allows for many-to-many relationships. As musicians